# Question of the day

- How did dynamic programming work again?
- How can a dynamic programming problem be solved iteratively, when dynamic programming is so strongly related to recursion?
- What's an algorithm design strategy that's applicable to more problems than divide-and-conquer, and more efficient and much easier to implement than dynamic programming?
  - What's the catch?

# Dynamic programming and greedy algorithms

**William Hendrix**

*Lecture 8*

# Outline

- Review
  - Design strategies
  - Memoization
- Memoized algorithm analysis
- Iterative dynamic programming
- Greedy algorithms

# Algorithm design review

- **Brute force**
  - A.k.a., exhaustive search
  - Try every possible solution
  - Applicable to all problem
  - Often unworkably slow
- **Divide-and-conquer**
  - Divide problem, solve recursively, and combine solutions
    - Reduce recursive calls if possible
  - Needs *optimal substructure*
    - Can efficiently compute full solution from recursive calls
  - Inefficient when subproblems overlap
- **Data organization**
  - Use appropriate data structures to improve complexity
  - Complements any other strategy
  - Time/space trade-off

# Dynamic programming review

- Algorithm design strategy related to divide-and-conquer
  - Make recursive calls, combine solutions, solve base cases directly
  - Recursive calls repeated => save solutions in lookup table
- Often reduces exponential algorithms to polynomial time
- Can be solved with *memoization* or iteratively
  - Function that stores return values in lookup table
  - Not limited to recursive functions/dynamic programming
- **Map** usually array-based
  - Dimensionality depends on parameters
  - Fibonacci(n): 1D array
  - Binomial(n, k): 2D array
- **Sentinel value**
  - Special value for a problem that's not yet been solved
  - Must not be a solution
  - E.g., -1 for Fibonacci or Binomial

# Memoization summary

- Five step procedure

1. **Start with naïve recursive algorithm**
   - Based on recurrence
2. **Decide data structure and sentinel value**
3. **Add "memoization check" to the beginning of the algorithm**
   - If solution is in data structure, return it
4. **Store solution before returning**
5. **Write wrapper function to initialize data structure**

$$\binom{n}{k} = \begin{cases} 1, \text{ if } k = 0 \text{ or } k = n \\ \binom{n-1}{k-1} + \binom{n-1}{k} \end{cases}$$

**Input:** $n$, $k$: binomial coefficient to compute
**Output:** $\binom{n}{k}$

```
Algorithm: DPBinom
binom = Array(n, k)
Initialize binom to 0
return MemoBinom(n, k)
```

```
Algorithm: MemoBinom
if binom[n, k] > 0 then
    return binom[n, k]
else if k = 0 or k = n then
    binom[n, k] = 1
else
    binom[n, k] = MemoBinom(n−1, k−1)
      + MemoBinom(n − 1, k)
end
return binom[n, k]
```

# Analyzing memoization

- LCS(air, care):

| | *a* | *i* | *r* | |
|---|---|---|---|---|
| **c** | *ar* | *r* | *r* | $\epsilon$ |
| **a** | *ar* | *r* | *r* | $\epsilon$ |
| **r** | | *r* | *r* | |
| **e** | | $\epsilon$ | $\epsilon$ | $\epsilon$ |
| | | $\epsilon$ | $\epsilon$ | |

$$LCS(a,b) = \begin{cases} a_1 + LCS(a',b'), & \text{if } a_1 = b_1 \\ \max(LCS(a',b), LCS(a,b')), & \text{otherwise} \end{cases}$$

<u>Two questions</u>

- How many cells are filled in (non-memoized calls)?
- How long does it take to fill in one cell?

**Total time** = # cells * Cost per cell + Initialization cost

- Could potentially require summation

# Analyzing our solution

- LCS(air, care):

| | *a* | *i* | *r* | |
|---|---|---|---|---|
| **c** | *ar* | *r* | *r* | $\epsilon$ |
| **a** | *ar* | *r* | *r* | $\epsilon$ |
| **r** | | *r* | *r* | |
| **e** | | $\epsilon$ | $\epsilon$ | $\epsilon$ |
| | | $\epsilon$ | $\epsilon$ | |

$$LCS(a,b) = \begin{cases} a_1 + LCS(a',b'), & \text{if } a_1 = b_1 \\ \max(LCS(a',b), LCS(a,b')), & \text{otherwise} \end{cases}$$

<u>Worst-case analysis</u>

- *O(nm)* cells
- *Θ(1)* time per cell
- Total time: *O(nm)*

# Iterative dynamic programming

## Main idea

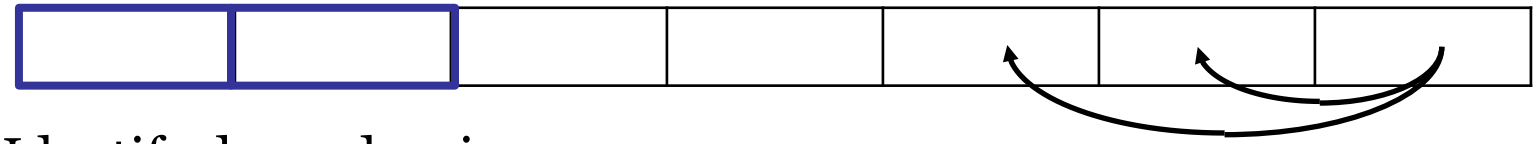
- Recursion not needed if dependent cells already filled in

<u>Process</u>

- Analyze dependencies to choose appropriate order
- Write for loop(s)
- Inside loop(s): naïve recursive algorithm
  - Replace calls with lookups
  - Replace return with assignment
  - Change variable to loop variable
- Allocate data structure beforehand

# Iterative example

- Fibonacci numbers
  - Each value is sum of two previous values
- **Recurrence:** $F(n) = F(n-1) + F(n-2)$
- One parameter
  - 1D array-based map



- Identify dependencies
  - Each cells needs the two cells to the left
  - Move in *opposite* direction
- Base cases
  - Must start here
- Need to iterate left-to-right
  - When calculating $F(n)$,

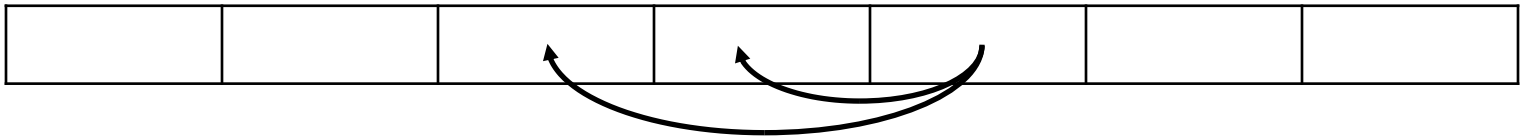
    $F(n\text{-}1)$ and $F(n\text{-}2)$ already solved

**Input:** $n$: index of Fibonacci number to compute
**Output:** $F_n$

```
Algorithm: IterativeFib
fib = Array(n)
for i = 1 to n do
    if i = 1 or 2 then
        fib[i] = 1
    else
        fib[i] = fib[i − 1] + fib[i − 2]
    end
end
return fib[n]
```

# Optimizing iterative DP

- Sometimes possible to improve space complexity
- **<u>Main idea</u>**
  - Don't store entire data structure
  - Only store what's necessary to calculate future values
- **Example:** Fibonacci numbers



- Each cell only needs previous two values
  - Values not useful after moving three away
- *Optimization:* only store previous two values
  - Constant space complexity

**Input:** $n$: index of Fibonacci number to compute
**Output:** $F_n$
**Algorithm:** BestFib

```
if n = 1 or 2 then
    return 1
end
prev = 1
twoprev = 1
for i = 3 to n do
    curr = prev + twoprev
    twoprev = prev
    prev = curr
end
return curr
```

# Iterative DP example

- Longest common subsequence

$$LCS(a,b) = \begin{cases} \epsilon, & \text{if } a = \epsilon \text{ or } b = \epsilon \\ a_1 + LCS(a', b'), & \text{if } a_1 = b_1 \\ \max(LCS(a', b), LCS(a, b')), & \text{otherwise} \end{cases}$$

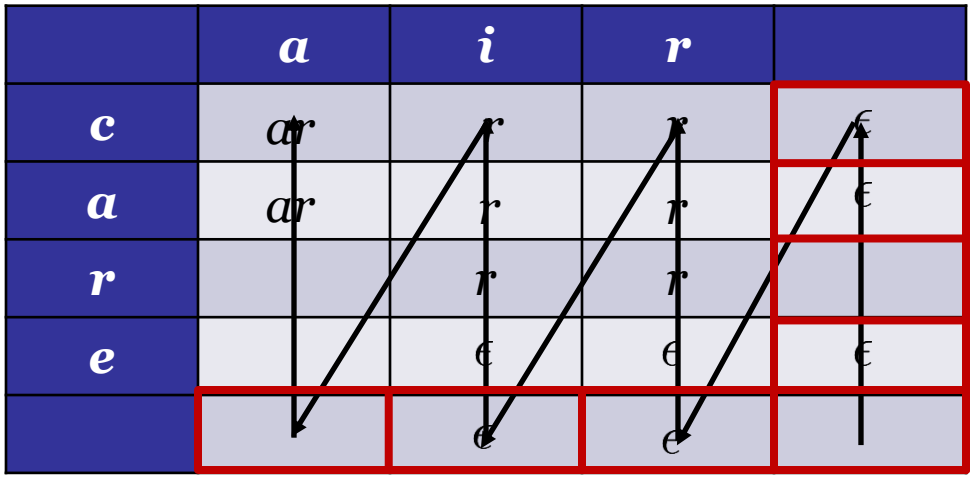
| | ***a*** | ***i*** | ***r*** | |
|---|---|---|---|---|
| ***c*** | | | | |
| ***a*** | | | | |
| ***r*** | | | | |
| ***e*** | | | | |
| | | | | |

- What order should we iterate through the array?

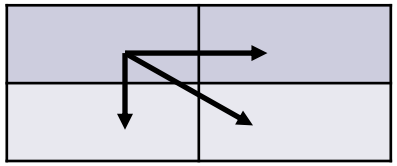
# Iterative DP example

- Longest common subsequence

$$LCS(a,b) = \begin{cases} \epsilon, & \text{if } a = \epsilon \text{ or } b = \epsilon \\ a_1 + LCS(a', b'), & \text{if } a_1 = b_1 \\ \max(LCS(a', b), LCS(a, b')), & \text{otherwise} \end{cases}$$




*Dependency structure:*



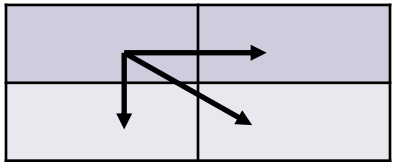
- What order should we iterate through the array?
  - Base cases are on bottom and right side of array
  - Cells to the right, bottom, and diagonal must be filled
  - Bottom-to-top, right-to-left works well
  - Other orders are possible

# Space complexity

- Can also reduce space complexity for LCS
- **Example**

| | *a* | *i* | *r* | |
|---|---|---|---|---|
| *c* | *ar* | *r* | *r* | $\epsilon$ |
| *a* | *ar* | *r* | *r* | $\epsilon$ |
| *r* | | *r* | *r* | |
| *e* | | $\epsilon$ | $\epsilon$ | $\epsilon$ |
| | | $\epsilon$ | $\epsilon$ | |

*Dependency structure:*

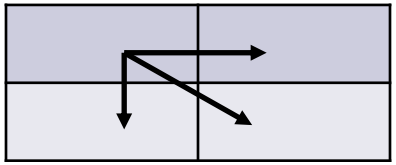


# Space complexity

- Can also reduce space complexity for LCS
- **Example**

| | *a* | *i* | *r* | |
|---|---|---|---|---|
| *c* | *a r* | *r* | *r* | $\epsilon$ |
| *a* | *a r* | *r* | *r* | $\epsilon$ |
| *r* | | *r* | *r* | |
| *e* | | $\epsilon$ | $\epsilon$ | $\epsilon$ |
| | | $\epsilon$ | $\epsilon$ | |

*Dependency structure:*



```
Input: a, b: strings to calculate LCS for
Input: n, m: length of a and b, respectively
Output: LCS of a and b
Algorithm: IterativeLCS
prev = Array(n + 1)
curr = Array(n + 1)
Initialize prev with ϵ
for i = n down to 1 do
    curr[m + 1] = ϵ
    for j = m down to 1 do
        if a[i] = b[j] then
            curr[j] = b[j] + prev[j + 1]
        else
            curr[j] = max{curr[j + 1], prev[j]}
        end
    end
    prev = curr
end
return curr[1]
```

Only need previous column

# Dynamic programming analysis

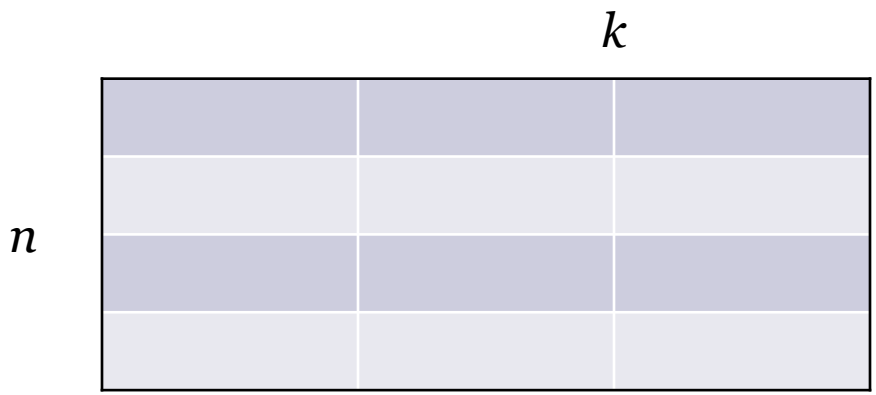
- Applies to recursive problems with overlap
- Generally reduces exponential-time to polynomial
- Increases space requirements
- Can be tricky

| | *a* | *i* | *r* | |
|---|---|---|---|---|
| *c* | *ar* | *r* | *r* | $\epsilon$ |
| *a* | *ar* | *r* | *r* | $\epsilon$ |
| *r* | | *r* | *r* | |
| *e* | | $\epsilon$ | $\epsilon$ | $\epsilon$ |
| | | $\epsilon$ | $\epsilon$ | |

- Memoization
  - Easier to program
  - May skip some cases
- Iteration
  - Often reduces memory requirements
  - Easier to analyze
  - Lower coefficients
    - Unless memoization skips too many cases

# Iterative DP exercise

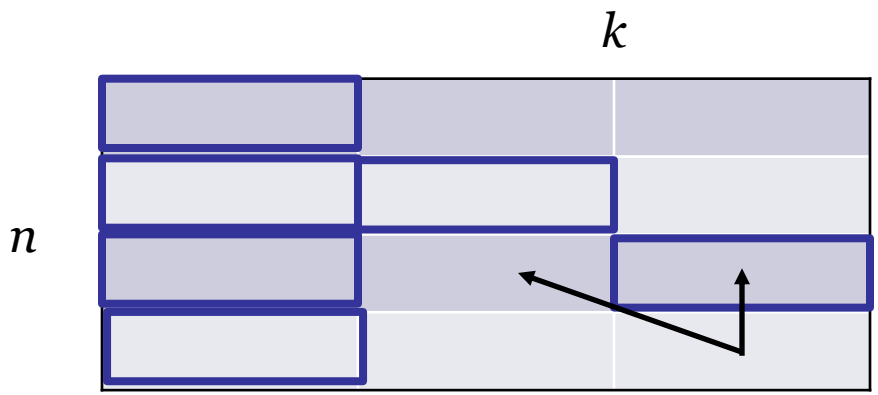
- Binomial coefficients ("$n$ choose $k$")
- **Recurrence:** $\binom{n}{k} = \begin{cases} 1, \text{ if } k = 0 \text{ or } k = n \\ \binom{n-1}{k-1} + \binom{n-1}{k} \end{cases}$




1. Where are base cases?
2. Which direction(s) do the recursive dependencies point?
3. What order should we iterate through array?
4. Describe for loop(s) that iterate in this order.
5. Give pseudocode for an iterative DP algorithm.

# Iterative DP exercise

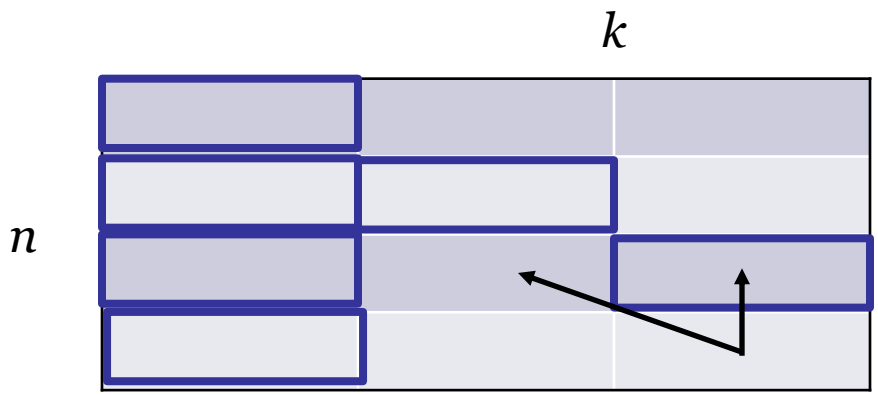
- Binomial coefficients ("$n$ choose $k$")
- **Recurrence:** $\binom{n}{k} = \begin{cases} 1, \text{ if } k = 0 \text{ or } k = n \\ \binom{n-1}{k-1} + \binom{n-1}{k} \end{cases}$




1. Where are base cases?
   k = 0 and n = k
2. Which direction(s) do the recursive dependencies point?
   Up and up-left

# Iterative DP exercise

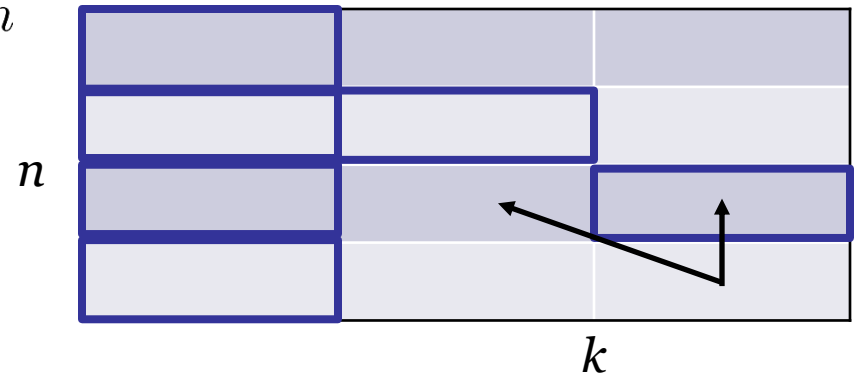
- Binomial coefficients ("$n$ choose $k$")
- **Recurrence:** $\binom{n}{k} = \begin{cases} 1, \text{ if } k = 0 \text{ or } k = n \\ \binom{n-1}{k-1} + \binom{n-1}{k} \end{cases}$




3. What order should we iterate through array?
4. Describe for loop(s) that iterate in this order.
5. Give pseudocode for an iterative DP algorithm.

# Iterative DP exercise

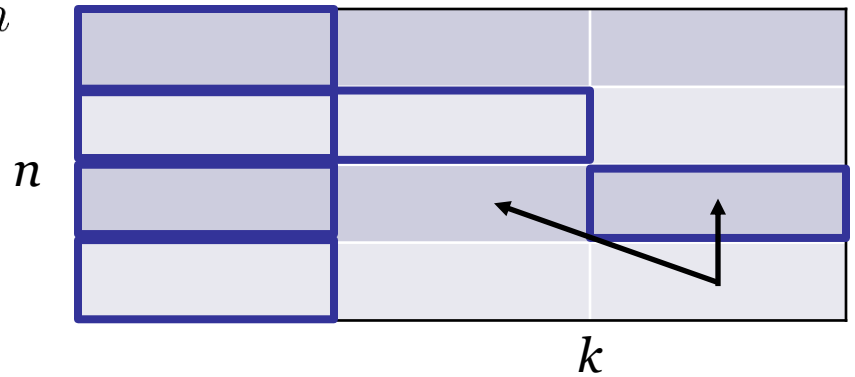
$$\binom{n}{k} = \begin{cases} 1, \text{ if } k = 0 \text{ or } k = n \\ \binom{n-1}{k-1} + \binom{n-1}{k} \end{cases}$$




3. What order should we iterate through the array?
4. Write for loop(s) that iterate in this order.
5. Give pseudocode for an iterative DP algorithm.

# Iterative DP exercise

$$\binom{n}{k} = \begin{cases} 1, \text{ if } k = 0 \text{ or } k = n \\ \binom{n-1}{k-1} + \binom{n-1}{k} \end{cases}$$




3. What order should we iterate through array?
   Multiple options
   E.g., left-to-right, top-to-bottom
4. Describe for loop(s) that iterate in this order.

```
for i = 0 to n
    for j = 0 to min{i, k}
```

```
for j = 0 to k
    for i = j to j + n - k
```

# Iterative DP exercise

**Input:** $n$: number of objects in total
**Input:** $k$: number of objects to pick
**Output:** number of ways to select $k$ objects from a pool of $n$

```
Algorithm: NChooseK
C = Array(n, k)
for i = 0 to n do
    for j = 0 to min{i, k} do
        if j = 0 or j = i then
            C[i, j] = 1
        else
            C[i, j] = C[i − 1, j − 1] + C[i − 1, j]
        end
    end
end
return C[n, k]
```

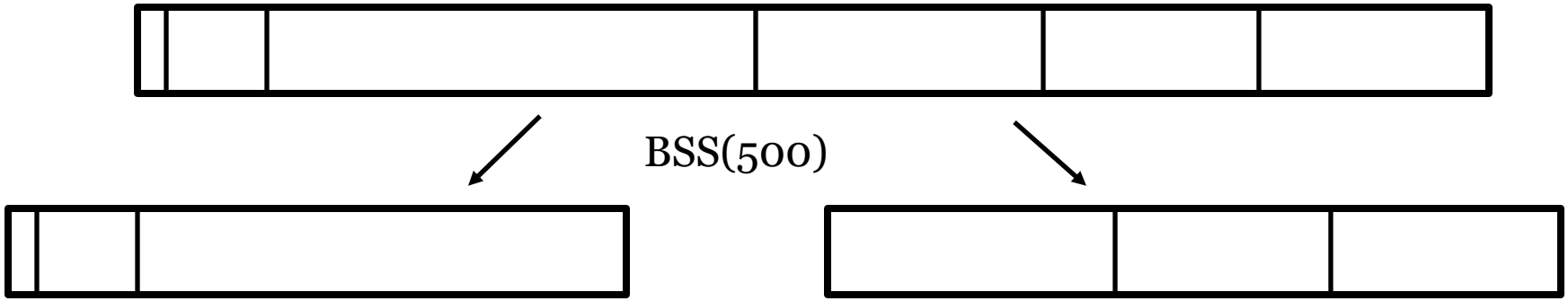
# Dynamic programming example

- Binary string splitting
  - Given a string *a* and index *i*, return *a[1..i]* and *a[i+1..n]*
  - Trivial, linear time
- Multiway string splitting
  - Use BSS to split string in multiple places
  - **Output:** order of cuts that minimizes cost
  - **Example:** *n=1000, i = (10, 100, 500, 700, 850)*

  - Left-to-right:
    - 10: cost = 1000
    - 100: cost = 990
    - 500: cost = 900
    - 700: cost = 500
    - 850: cost = 300
    - Total cost = 3690
  - Binary splitting:
    - 500: cost = 1000
    - 100: cost = 500
    - 700: cost = 500
    - 850: cost = 300
    - 10: cost = 100
    - Total cost = 2400

# Identifying optimal substructure

- Make a decision




- Can we use recursion to solve the rest?
  - Yes!
- How do we combine recursive solutions to solve overall problem?
  - For each decision, cost of cut + best cost(LHS) + best cost(RHS)
  - Best cost = min cost of all decisions
  - Base case
    - No cuts left to make
    - Cost = 0
- Do subproblems overlap?
  - Yes: cut at 500, then 700 yields same "pieces" as reverse order

# String splitting

- Write recurrence
  - Best cost = min{length + best cost(LHS) + best cost(RHS)}
  - $C(a) = n + \min_x \{C(a[1..x]) + C(a[x+1..n])\}$
  - $C(a[x..y]) = y - x + \min_i \{C(a[x..i]) + C(a[i..y])\}$
- Identify parameters of recursive function
  - String, cuts, $x$, $y$
- Data structure: $\text{cuts} \times \text{cuts}$
- Base case: no cuts between $x$ and $y$ (consecutive)

| | 10 | 100 | 500 | 700 | 850 | 1000 |
|---|---|---|---|---|---|---|
| 0 | | | | | | |
| 10 | | | | ? | | |
| 100 | | | | | | |
| 500 | | | | | | |
| 700 | | | | | | |
| 850 | | | | | | |

Final solution

# Putting it all together: memoization

$$C(a[\ell[x]..\ell[y]]) = \ell[y] - \ell[x] + \min_{x<i<y} \{C(a[\ell[x]..\ell[i]]) + C(a[\ell[i]..\ell[y]])\}$$

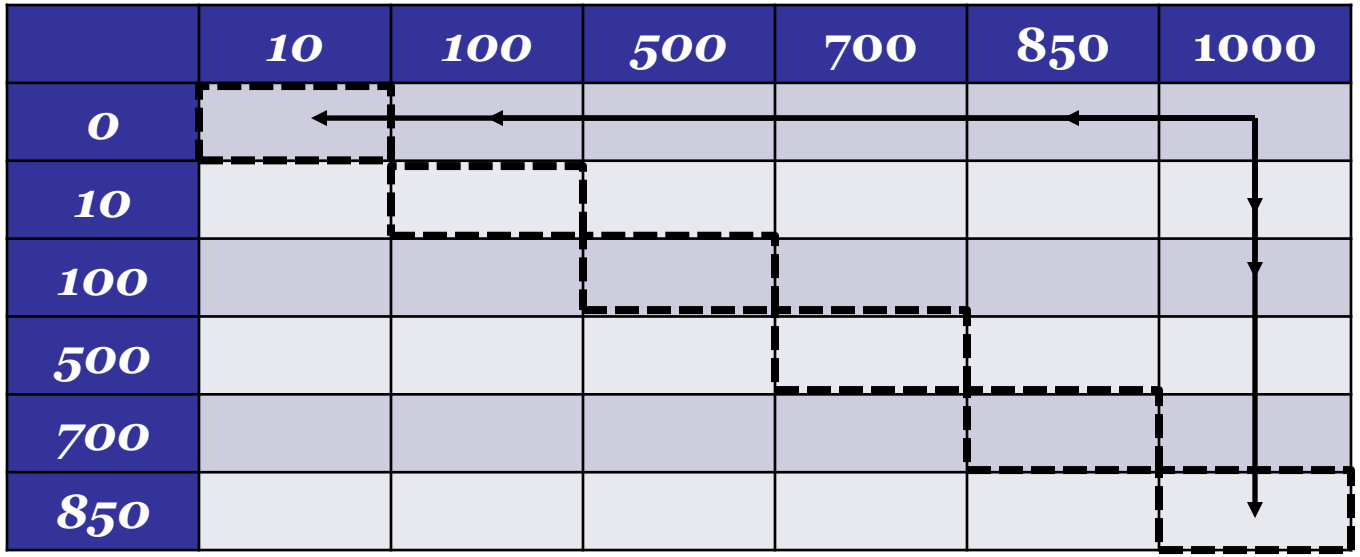
**Input:** $a$: string of length $n$
**Input:** $\ell$: locations at which to split $a$
**Input:** $k$: length of $\ell$
**Output:** Minimum cost of splitting $a$

**Algorithm:** MinSplit
$cost = \text{Array}(k+2, k+2)$
Initialize $cost$ to $-1$
Prepend 1 to $\ell$ and append $n$
**return** MemoSplit(1, $k+2$)

**Algorithm:** MemoSplit$(x, y)$
**if** $cost[x, y] \neq -1$ **then**
  **return** $cost[x, y]$
**else if** $y = x + 1$ **then**
  $cost[x, y] = 0$
**else**
  $mincost = \infty$
  **for** $z = x + 1$ to $y - 1$ **do**
    $temp = \ell[y] - \ell[x] + \text{MemoSplit}(x, z) + \text{MemoSplit}(z, y)$
    $mincost = \min\{mincost, temp\}$
  **end**
  $cost[x, y] = mincost$
**end**
**return** $cost[x, y]$

# Iterative version

- Analyze dependency structure




- Describe order that respects dependencies
  - E.g., bottom-to-top, left-to-right
- Write loops that iterate in this order
- Body of loop = recursive function without memoization

# Iterative pseudocode

```
Input: a: string of length n
Input: ℓ: locations in a to cut
Input: k: length of ℓ
Output: Minimum cost to split a at ℓ
Algorithm: IterSplit
Prepend 1 to ℓ
Append n to ℓ
cost = Array(k + 2, k + 2)
for δ = 1 to k + 1 do
    for x = 1 to k + 2 − δ do
        y = x + δ
        if y = x + 1 then
            cost[x, y] = 0
        else
            mincost = ∞
            for z = x + 1 to y − 1 do
                temp = ℓ[y] − ℓ[x] + cost[x, z] + cost[z, y]
                mincost = min{mincost, temp}
            end
            cost[x, y] = mincost
        end
    end
end
return cost[1, k + 2]
```

Diagonal loops
*Tricky!*

Could also make separate loop for base cases

Same as always

Complexity: $\Theta(k^3)$
Space: $\Theta(k^2)$
Not possible to reduce space complexity

# Iterative pseudocode

```
Input: a: string of length n
Input: ℓ: locations in a to cut
Input: k: length of ℓ
Output: Minimum cost to split a at ℓ
Algorithm: IterSplit
Add 1 and n to the beginning and end of ℓ
cost = Array(k + 2, k + 2)
for x = 1 to k + 1 do
    for y = x − 1 down to 1 do
        if y = x − 1 then
            cost[x, y] = 0
        else
            mincost = ∞
            for z = x + 1 to y − 1 do
                temp = ℓ[y] − ℓ[x] + cost[x, z] + cost[z, y]
                mincost = min{mincost, temp}
            end
            cost[x, y] = mincost
        end
    end
end
return cost[1, k + 2]
```

Could also make separate loop for base cases

Recursive case

Complexity: $\Theta(k^3)$
Space: $\Theta(k^2)$
Not possible to reduce space complexity

# Strategy: greedy algorithms

- Usually applied to *optimization* problems
  - "Find the best/largest/smallest/etc ..."

- **Strategy outline**
  - Formulate the problem as sequence of decisions
    - E.g., which element to add to/remove from a set
  - Always select the best available option
    - "Best" depends on problem
      - "Local optimum"
  - Continue until solution cannot be improved

# Greedy example

- **Problem:** knapsack problem
- **Input:** array of positive integers *data* (size $n$), and target weight $t$
- **Output:** largest subset of data with sum $\leq t$

- **Example:** $data = \{2, 3, 8, 7, 7, 6\}$, $t = 21$

# Greedy example

- **Problem:** knapsack problem
- **Input:** array of positive integers *data* (size $n$), and target weight $t$
- **Output:** largest subset of data with sum $\leq t$

- **Example:** *data* = {2, 3, 8, 7, 7, 6}, $t$ = 21
  - *Solution:* {2, 3, 6, 7} or {2, 3, 6, 8}

1. What is our sequence of decisions?
   - What number do we add next?
2. Which option is best?
   - Add the smallest remaining element

# Example greedy algorithm

```
Input: data: array of positive integers
Input: n: size of data
Input: t: target value
Output: largest subset of data with sum ≤ t
Algorithm: GreedyKnapsack
heap = Heapify(data)
soln = {}
sum = 0
next = heap.DeleteMin()
while sum + next ≤ t do
    Add next to soln
    sum = sum + next
    next = heap.DeleteMin()
end
return soln
```

- Alternatively: sort first, then iterate
  - Same answer and time complexity
- Is the answer correct?

# Greedy correctness

- Proof of correctness is usually via contradiction
  - Or counterexample when incorrect

- Proof outline:
  1. Take an arbitrary solution *not* produced by greedy algorithm
  2. Make it more like greedy solution *without* reducing quality
     - Find first "divergence"
- *Informally:* any solution can be transformed into greedy solution

- Underlying idea:
  1. Let $X$ be greedy solution you produced
  2. Let $Y$ be a better solution as similar as possible to $X$
  3. Find new solution $Y'$ that is just as good as $Y$ and more similar to $X$
  4. *Contradiction!*

# GreedyKnapsack proof sketch

- Let $X$ be greedy solution
- Let $Y$ be a maximum subset with sum $\leq t$
- Let $x$ be the smallest element of $X - Y$
- Let $y$ be the smallest element of $Y - X$
- If $y \leq x$, GK would have added $y$ to $X$
    - $x < y$
- Add $x$ and remove $y$ from $Y$
  - Same size as $Y$
  - Sum $< t$
  - More overlap with $X$

# Formal proof of correctness

*Proof.* We prove the claim by contradiction. Suppose that GreedySearch (by earliest end time) is not correct. Let start and duration represent an instance that GreedySearch solves incorrectly, let *bestset* be the set with the largest number of workshops, and let $found$ be the set identified by GreedySearch.

Since $found$ is incorrect, *bestset* must have at least one element more than $found$. As a result, it must contain a workshop that $found$ does not. We use $x$ to denote the earliest workshop that is in *bestset* but not $found$. Since $x$ is not in $found$, there must be some workshop $y$ in $found$ that overlaps $x$. Because of how GreedySearch operates, $y$ must have an earlier ending time than $x$.

Consider the set formed by removing $x$ from *bestset* and adding $y$. Since all of the workshops that end before $x$ in *bestset* are also in $found$ and $found$ contains no overlaps, $y$ cannot overlap any earlier workshop in *bestset*. Also, since $y$ has an earlier end time than $x$ and $x$ does overlap any later workshop in *bestset*, $y$ cannot either. So, this new set contains no overlaps and the same number of workshops as *bestset*. Thus, we can construct a solution of the same size that contains $y$.

If we repeat this process, we can eliminate every element ($y$) of *bestset* not in $found$ without changing its size. This is impossible, as the resulting set would contain only the elements of $found$, but it would still have more elements than found. $\Rightarrow\Leftarrow$

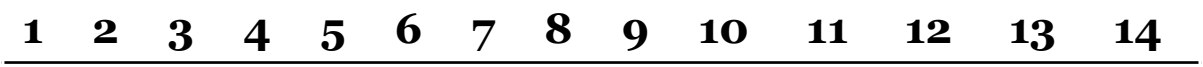
Thus, our assumption that a larger solution exists must not be correct. $\square$

# Algorithm design example

- Consider the following problem:
- **Problem:** workshop scheduling
  - **Input:** the start times and durations for a set of workshops
  - **Output:** the largest number of workshops whose times do not overlap

- **Example instance:**

| | Start | Dur. |
|---|---|---|
| A | 1 | 7 |
| B | 2 | 3 |
| C | 3 | 6 |
| D | 6 | 3 |
| E | 8 | 2 |
| F | 9 | 3 |
| G | 9 | 5 |
| H | 11 | 1 |
| I | 13 | 1 |



# GreedySearch

**Input:** *start*: an array of start times for workshops
**Input:** *duration*: an array of durations for workshops
**Input:** $n$: the number of workshops
**Output:** Maximum possible workshops to attend

```
Algorithm: GreedyWorkshop
best = 0
subset = {}
Sort start and duration by _____
while start and duration not empty do
    Add (start[1], duration[1]) to subset
    Remove all workshops from start and duration
      that overlap this workshop
    best = best + 1
end
return best
```

# GreedySearch variants

- Earliest workshop first

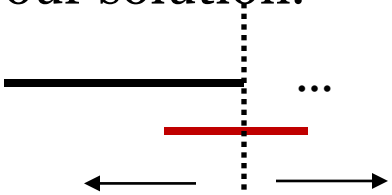
- Shortest workshop first

# GreedySearch variants

- Earliest workshop first
- **Counterexample:**

- Shortest workshop first
- **Counterexample:**

- Earliest workshop end first
  - Guaranteed to be correct
  - **Proof idea:** when we select the first workshop to add, every workshop we eliminate must overlap everything else we eliminate, and possibly more workshops. None of these could improve our solution.




Starts before here  Ends after here

# Coming up

- Greedy algorithms
- Graph representations
- Graph traversals

- **Recommended readings:** Sections 12.2, 12.5-12.5.2
- *Practice problems:* R-12.8, C-12.2 and C-12.4 (read section 12.4), C-12.6, A-12.1